第14课 二维数组

> 本讲义配合课堂例题讲解使用，代码可直接在 Dev-C++ 中运行验证。

五、课堂例题

例题 1：输入一个矩阵并原样输出（入门）

题目：输入两个整数 n 和 m（n≤5，m≤5），再输入一个 n 行 m 列的整数矩阵，然后原样输出这个矩阵（每行的数之间用空格隔开）。

讲解思路：

> 这道题分两步：先把数据"搬进"二维数组，再从二维数组里"搬出来"显示。两步都用双重循环。输出的时候注意每行末尾要换行，每行的数之间用空格隔开——我们可以在每个数后面输出一个空格，简单又好记。

完整代码：

```
#include <iostream>
using namespace std;

int main() {
    int n, m;
    cin >> n >> m;

    int a[5][5]; // 开足够大的数组

    // 输入：双重循环，外层行、内层列
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < m; j++) {
            cin >> a[i][j];
        }
    }

    // 输出：同样的双重循环
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < m; j++) {
            cout << a[i][j];
            if (j < m - 1) cout << " "; // 最后一列后面不加空格
        }
        cout << endl; // 每行结束换行
    }

    return 0;
}
```

示例运行：

```
输入：
2 3
1 2 3
4 5 6

输出：
1 2 3
4 5 6
```

讲解要点：
- 数组声明时大小要**开够**：题目说 n≤5、m≤5，所以 `a[5][5]` 就够了。
- 输入和输出的双重循环结构**一模一样**，只是里面做的事不同——一个是 `cin`，一个是 `cout`。
- `if (j < m - 1)` 用来控制空格，避免行末多出一个空格。可以告诉学生"这是输出格式的小技巧"。

例题 2：输出乘法表格（进阶理解）

题目：输入一个整数 n（n≤9），输出一个 n×n 的乘法表格。第 i 行第 j 列的值为 (i+1)×(j+1)。每个数占 4 个宽度，右对齐。

讲解思路：

> 这道题不需要输入数据到数组里，而是直接用双重循环**计算并输出**。重点是让同学们理解"下标从 0 开始"和"实际数值从 1 开始"之间的转换关系。

完整代码：

```
#include <iostream>
#include <iomanip>
using namespace std;

int main() {
    int n;
    cin >> n;

    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            cout << setw(4) << (i + 1) * (j + 1);
        }
        cout << endl;
    }

    return 0;
}
```

示例运行：

```
输入：
4

输出：
   1   2   3   4
   2   4   6   8
   3   6   9  12
   4   8  12  16
```

讲解要点：
- `setw(4)` 是设置输出宽度为 4，需要 `#include <iomanip>`。告诉学生"这就像在表格的每个格子里写数字，每个格子一样宽，数字靠右边写"。
- `(i + 1) * (j + 1)` —— 因为下标从 0 开始，但乘法表从 1 开始，所以要 +1。强调"下标 vs 实际值"的区别。
- 这道题没有用数组存储，直接计算输出。可以提问学生："如果我后面还想用这些数，该怎么办？"——引导出"先存再用"的思路。

例题 3：成绩表——求每行之和与每列之和（应用）

题目：有 n 个同学（n≤5），每人有 m 科成绩（m≤5）。输入所有成绩后，分别输出：
1. 每个同学的总分
2. 每科的平均分（保留1位小数）

讲解思路：

> 还记得老师说的"你是班长帮老师统计成绩"的故事吗？现在我们来真的！
>
> - "每个同学的总分"就是**每一行的所有数加起来**——固定行号，遍历列。
> - "每科的平均分"就是**每一列的所有数加起来再除以人数**——固定列号，遍历行。
>
> 关键是搞清楚：**求行和时，外层循环是行；求列和时，外层循环是列**。

完整代码：

```
#include <iostream>
#include <iomanip>
using namespace std;

int main() {
    int n, m;
    cin >> n >> m;

    int a[5][5];

    // 输入成绩
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < m; j++) {
            cin >> a[i][j];
        }
    }

    // 求每个同学的总分（每行之和）
    cout << "每个同学的总分: " << endl;
    for (int i = 0; i < n; i++) {
        int sum = 0;
        for (int j = 0; j < m; j++) {
            sum += a[i][j];
        }
        cout << "同学" << i + 1 << "：" << sum << endl;
    }

    // 求每科的平均分（每列之和 / 人数）
    cout << "每科的平均分: " << endl;
    for (int j = 0; j < m; j++) {
        int sum = 0;
        for (int i = 0; i < n; i++) {
            sum += a[i][j];
        }
        double avg = (double)sum / n;
        cout << "第" << j + 1 << "科: " << fixed << setprecision(1) << avg << endl;
    }

    return 0;
}
```

示例运行：

```
输入：
3 3
90 85 78
76 92 88
85 79 95

输出：
每个同学的总分：
同学1：253
同学2：256
同学3：259
每科的平均分：
第1科：83.7
第2科：85.3
第3科：87.0
```

讲解要点：
- **求行和**：外层 `i` 控制行（哪个同学），内层 `j` 遍历列（每一科），`sum` 在每行开始时归零。
- **求列和**：外层 `j` 控制列（哪一科），内层 `i` 遍历行（每个同学），注意循环变量的角色互换了！
- `(double)sum / n`：强制类型转换，否则整数除以整数会丢掉小数部分。可以用生活比喻："3 块饼干分给 2 个人，如果用整数除，每人只得 1 块，那半块饼干就'消失'了！"
- `fixed << setprecision(1)`：控制输出 1 位小数。

附：例题源代码与运行结果

例题 1：Input and Output a Matrix

```
// Input n rows and m columns of integers, then print the matrix as-is.
#include <iostream>
using namespace std;

int main() {
    int n, m;
    cin >> n >> m;

    int a[5][5];

    // Input: nested loop, outer for rows, inner for columns
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < m; j++) {
            cin >> a[i][j];
        }
    }

    // Output: same nested loop structure
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < m; j++) {
            cout << a[i][j];
            if (j < m - 1) cout << " ";  // no trailing space
        }
        cout << endl;  // newline after each row
    }

    return 0;
}
```

Sample Input

```
2 3
1 2 3
4 5 6
```

Expected Output

```
1 2 3
4 5 6
```

例题 2：Multiplication Table Grid

```
// Input n, output an n*n multiplication table.
// Value at row i, col j is (i+1)*(j+1). Each number is right-aligned, width 4.
#include <iostream>
#include <iomanip>
using namespace std;

int main() {
    int n;
    cin >> n;

    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            cout << setw(4) << (i + 1) * (j + 1);
        }
        cout << endl;
    }

    return 0;
}
```

Sample Input

```
4
```

Expected Output

```
1   2   3   4
   2   4   6   8
   3   6   9  12
   4   8  12  16
```

例题 3：Grade Table - Row Sum and Column Average

```
// n students, m subjects each. Compute:
// 1) Each student's total score (row sum)
// 2) Each subject's average score (column average, 1 decimal place)
#include <iostream>
#include <iomanip>
using namespace std;

int main() {
    int n, m;
    cin >> n >> m;

    int a[5][5];

    // Input scores
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < m; j++) {
            cin >> a[i][j];
        }
    }

    // Row sums: total score per student
    for (int i = 0; i < n; i++) {
        int sum = 0;
        for (int j = 0; j < m; j++) {
            sum += a[i][j];
        }
        cout << "Student " << i + 1 << ": " << sum << endl;
    }

    // Column averages: average score per subject
    for (int j = 0; j < m; j++) {
        int sum = 0;
        for (int i = 0; i < n; i++) {
            sum += a[i][j];
        }
        double avg = (double)sum / n;
        cout << "Subject " << j + 1 << ": "
             << fixed << setprecision(1) << avg << endl;
    }

    return 0;
}
```

Sample Input

```
3 3
90 85 78
76 92 88
85 79 95
```

Expected Output

```
Student 1: 253
Student 2: 256
Student 3: 259
Subject 1: 83.7
Subject 2: 85.3
Subject 3: 87.0
```